

Module 3

Vibe Coding and Natural-Language-Driven Development

**Module 3 does not replace Module 2.
It stress-tests it.**

Generate a first draft faster. Then review, verify, and own it.

Continuation from Module 2

Same ownership principle, broader first-draft delegation

Module 2

How do I use AI inside a disciplined developer workflow?

Frame → Plan → Prompt → Execute → Verify → Reflect

Module 3

What happens when AI generates more of the first draft from product intent?

Brief → Explore → Contain → Generate → Review → Own

The difference is entry point and delegation size — not accountability.

- ▶ Module 2 habits become more important, not less important.
- ▶ The first draft is faster; the handoff burden is higher.
- ▶ A polished UI is not proof of maintainable code.

Learning outcomes

What students should be able to do after this module

- ❑ Define vibe coding in a disciplined developer context
- ❑ Use Brief → Explore → Contain → Generate → Review → Own
- ❑ Write broad Stage A prompts and narrow them safely
- ❑ Identify good-fit and bad-fit tasks
- ❑ Review and repair an AI-generated first draft
- ❑ Connect the frontend slice to the Module 2 API
- ❑ Explain what they own after the first draft

Practical anchor

Task Tracker frontend: local-first first draft, then minimal end-to-end integration with the Module 2 Work Items API.

Do not skip

handoff problem · verification · ownership · API integration

Safe to skim

tool comparison · local/self-hosted setup · visual polish

What do we mean by “vibe coding”?

A useful term only if we define its boundaries

A conversational, natural-language-driven workflow where the developer starts closer to product intent and lets AI take a larger share of first-draft implementation.

It is

- first-draft acceleration
- rapid exploration
- visible prototypes
- human-guided narrowing

It is not

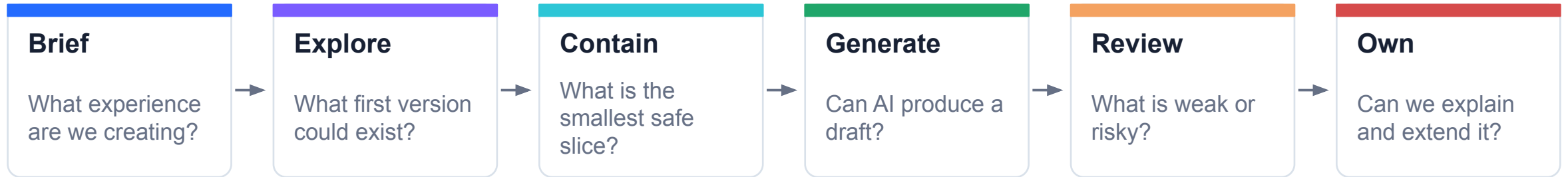
- no requirements
- no tests
- no review
- shipping the first draft

Key phrase

- The vibe is in the first draft. The professionalism is in what happens after.

The workflow spine

Brief → Explore → Contain → Generate → Review → Own



**This is not “prompt once and ship.”
It is a controlled funnel from intent to owned code.**

Module 2 vs. Module 3 in one example

Same task tracker theme, different entry point

Module 2 prompt style

“Implement repository filtering for status and priority. Keep API response shape stable. Add tests.”

Module 3 prompt style

“Based on this product brief, propose two useful first-version task-tracker experiences. Do not edit files yet.”

Module 2 risk

wrong implementation or weak tests

Module 3 risk

beautiful draft that no one owns

Module 3 increases speed. It also increases the ownership gap.

The three-stage prompt progression

Start broad, then narrow, then own

Stage A · Explore

Ask for user-flow options, omissions, and tradeoffs.
No edits yet.

Stage B · Contain

Choose the smaller slice. Define files, constraints, stop conditions, and checks.

Stage C · Own

Ask for handoff risks, maintainability issues, tests, and manual review.

Stage A is allowed to be broad.

Stage B must become concrete.

Stage C turns generated code into team-owned code.

Stage A: Explore product intent

Broader than Module 2, but still read-only

Opening prompt

Read docs/design-brief.md.

Propose two useful first-version experiences for this task tracker.

Focus on visible user flow, core behavior, and what should be omitted.

Do not edit files yet.

What to look for

- two options
- tradeoffs
- omissions
- verification cost
- classroom-safe scope

Reject early

frameworks, auth, drag-and-drop, charts, dark mode, new dependencies

Stage B: Contain the first slice

The decisive move from broad idea to reviewable work

- ▶ Choose the smaller direction
- ▶ List files to touch and files to avoid
- ▶ Define stop conditions
- ▶ State tests and manual browser checks
- ▶ Ban new dependencies unless deliberately approved
- ▶ Keep API integration minimal

Containment is how you keep a vibe-coded first draft from becoming a mystery app.

Classroom stop condition

Stop after render, quick-add, one status filter, completion toggle, local persistence, and empty state.

Stage C: Own the generated result

The handoff starts immediately after the first draft

Ask for handoff risks

file responsibilities · assumptions · weak tests · unsafe DOM
· accessibility · API errors

Perform one cleanup

small, reviewable fix that improves explainability or safety

Explain the code

what each file does · where state lives · where API boundary lives · how to verify

Record ownership

what AI generated well · what you rejected · remaining risks · next human-owned change

When vibe coding works well — and when it backfires

Fit depends on blast radius and verification cost

Good fit

- greenfield UI slices
- CRUD-heavy internal tools
- dashboards and admin panels
- prototypes and demos
- cheap manual verification

Bad fit

- auth and permissions
- payments or billing
- production migrations
- regulated data
- hard-to-test behavior
- unclear requirements

Vibe coding is safest when the first draft is visible, bounded, and easy to throw away.

Conversational coding agents and Devin-style workflows

Teach the workflow pattern, not a specific subscription

Conversational vibe coding

- frequent steering
- IDE/chat-centered
- visible first drafts

IDE / terminal agent

- bounded local edits
- permissions matter
- review via diff/tests

Issue-driven agent

- less continuous steering
- branch / PR / report
- review after longer work

Less steering does not mean less review. It usually means more review.

Tool fit by workflow shape

OPTIONAL

Use this only if the cohort benefits from tool comparison

IDE-centered classroom baseline

Copilot

Terminal-first plan/edit permissions

Claude Code

Parallel, cloud, frontend/browser QA or PoC work

Codex

Local/offline/model-controlled workflow

Continue.dev or OpenHands

The durable skill is choosing the right autonomy level, not memorizing product UI.

Optional: local or self-hosted vibe coding

OPTIONAL

Good reasons, real tradeoffs

Good reasons

- data boundaries
- model control
- limited internet
- procurement constraints
- training environments

Tradeoffs

- setup complexity
- slower iteration
- hardware requirements
- maintenance
- security ownership

Do not let tool setup consume the core lesson: ownership of generated code.

The AI handoff problem

The real heart of the module

- ▶ Can we explain this code?
- ▶ Can we maintain it?
- ▶ Can we extend it safely?
- ▶ Do we know the stable boundaries?
- ▶ Where is the safe seam if we regenerate later?
- ▶ Which tests prove behavior?

Working UI is not yet owned software.

Ownership evidence

- clear file responsibilities
- tests + manual checks
- API boundary
- error handling
- rejected suggestions
- reflection

Existing codebases: add Stage 0

Read-only orientation before broad generation

Stage 0

Map the current app first. Identify seams, conventions, dependencies, files to avoid.

Then Stage A

Explore first-version experiences within those boundaries.

Then Stage B/C

Contain, generate, review, and own.

- ▶ Do not ask AI to redesign a mature app from a broad prompt.
- ▶ Ask for a map first.
- ▶ Preserve established boundaries and conventions.
- ▶ Make regeneration seams explicit.

Frontend design and visual generation

OPTIONAL

Keep it developer-centric

AI can help

- layout options
- microcopy
- empty states
- component inventories
- screenshot-to-code experiments

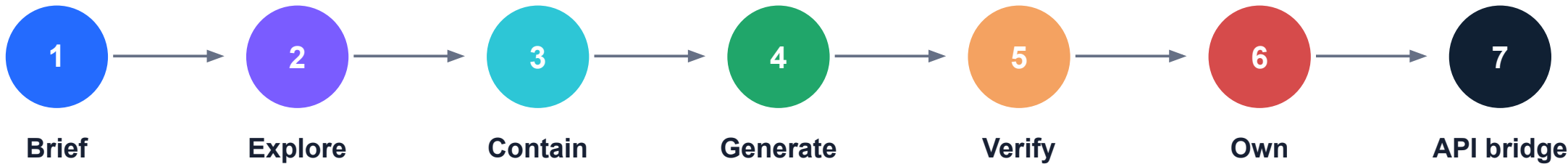
Still review

- accessibility
- semantics
- state flow
- brittle CSS
- unsafe markup
- maintainability

Visual polish is not ownership.

Guided demo: first draft → owned slice

Task Tracker frontend, then Module 2 API bridge



Demo success

Students see decisions: what to accept, what to reject, what to verify, and what to own.

Lab Phase 1: vibe-code the first version

Core slice first; stretch only if there is time

Core required

- render tasks
- quick-add
- one status filter
- toggle done
- local persistence
- empty state

Optional stretch

- search
- priority editing
- combined filters
- responsive polish
- keyboard polish
- dark mode

A smaller owned slice beats a bigger mystery app.

Review the first draft like a skeptical owner

Generated code deserves a handoff review

Some ideas: **

- Unsafe DOM rendering of task text
- Monolithic file responsibilities
- Corrupt localStorage handling
- Weak or missing tests
- Status mismatch with backend API
- Accessibility and focus gaps
- Dependency or framework creep

Use the flawed draft exercise

`review_exercises/flawed_generated_draft/`

The answer can look clean and still be unsafe or unowned.

Lab Phase 2: ownership and API integration

Required bridge to the Module 2 Work Items API

Ownership cleanup

- explain files
- fix one risk
- improve one UX/accessibility issue
- strengthen verification

API integration

- list
- create
- toggle todo/done
- update priority
- show API unavailable error
- keep local fallback

This is where Module 3 becomes genuinely end-to-end.

Optional extensions for faster cohorts

OPTIONAL

Only after the required ownership work is complete

free-text search

Ask for two approaches. Choose the smaller one. Add tests or manual checks.

detail panel

Ask for two approaches. Choose the smaller one. Add tests or manual checks.

due-date display

Ask for two approaches. Choose the smaller one. Add tests or manual checks.

keyboard focus polish

Ask for two approaches. Choose the smaller one. Add tests or manual checks.

dark mode

Ask for two approaches. Choose the smaller one. Add tests or manual checks.

priority sorting

Ask for two approaches. Choose the smaller one. Add tests or manual checks.

Homework

OPTIONAL

Every exercise includes a suggested solution

1. Ownership review

Review flawed generated draft and write findings.

2. Safe optional feature

Add one feature with tests/checks and reflection.

3. Reflection

Vibe-to-ownership one-page write-up.

Reward caught mistakes, not only visible features.
Require one rejected or revised AI suggestion.
Require verification evidence.

Takeaways and bridge to Module 4

Where natural-language generation ends and ownership begins

- ▶ Vibe coding is useful for first drafts and exploration.
- ▶ Broad prompts must quickly become bounded plans.
- ▶ Visual polish is not ownership.
- ▶ Generated code needs explanation, tests, review, and cleanup.
- ▶ Existing codebases require read-only orientation first.
- ▶ The Module 2 workflow still applies after the draft exists.

The real skill is learning when acceleration ends and engineering ownership begins.

Bridge to Module 4

Next: apply the same discipline across the product lifecycle — requirements, planning, delivery, and organizational workflow.